

贪心算法

说到贪心算法，想必诸位也不陌生。我们已经学过很多贪心算法了，例如 Dijkstra 算法以及最小生成树的两种算法。在这些算法中，我们的任务是要得到一个最优的结果，于是每一步都选取满足某些性质的最好结果（例如 Dijkstra 每一步都找目前能找到的最近点）。如果能证明这些局部最优就是全局最优，那么贪心算法就是正确的。当然很可能贪心算法不一定能带来最优结果，那么这时可能要更换策略，或者本身问题就很难得到最优解，那么贪心作为一种高效率的启发式算法，可以尝试计算一个贪心算法能和最优解之间的差的最大比例（将在近似算法一节讨论）。接下来将讨论几个经典的贪心算法的应用，讨论设计贪心算法的关键以及证明其正确性的思路。

9.1 活动选择问题

假如你负责安排学校的社团活动，现在有若干个社团申请使用同一间教室，每场活动的开始时间和结束时间都已经确定，不能调整。教室在同一时间只能供一场活动使用，因此有些申请无法同时满足。你希望让尽可能多的活动得到场地，应该接受哪些申请呢？这就是**活动选择问题**。

将这些活动记为 $S = \{a_1, a_2, \dots, a_n\}$ 。每个活动 a_i 的开始时间为 s_i ，结束时间为 f_i ，且 $0 \leq s_i < f_i < \infty$ 。我们允许一个活动结束时，下一个活动立即开始，因此用半开区间 $[s_i, f_i)$ 表示活动占用的时间。若 $f_i \leq s_j$ 或 $f_j \leq s_i$ ，两个活动就不会占用同一段时间，称它们是**兼容的**。题目要求选出尽可能多的活动，使其中任意两个都兼容。

假设输入是按照 n 个活动结束时间从小到大排列的。在学完动态规划后，相信读者应该对这一问题并不陌生——这里又遇到了类似于加权独立集合中有冲突的情况，所以我们可以设计出如下两种动态规划的递推式来解决这一问题：

1. 先考虑在活动 a_i 结束后、活动 a_j 开始前，最多还能安排多少个活动。将可以放在这段时间内的活动组成集合

$$S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\},$$

用 c_{ij} 表示从中能选出的最多兼容活动数。如果决定选择其中的活动 a_k ，其他所选活动就必须要在它开始前结束，或在它结束后开始，分别属于 S_{ik} 和 S_{kj} ，不难得到

$$c_{ij} = \max_{a_k \in S_{ij}} \{c_{ik} + c_{kj} + 1\}. \quad (9.1)$$

当然为了能使用这一递推式，我们要再加上虚拟活动 a_0 和 a_{n+1} ，规定 $f_0 = 0$ 、 $s_{n+1} = +\infty$ ，它们只用于表示所有实际活动之前和之后的时间，不计入所选活动数，这样原问题的答案就是 $c_{0,n+1}$ 。

2. 也可以按最后一个活动选不选来分类。令 c_i 表示前 i 个活动中最多能选出的兼容活动数。不选 a_i 时，答案为 c_{i-1} ；选 a_i 时，其他活动必须在它开始前或恰在它开始时结束。将满足 $f_k \leq s_i$ 的

最后一个活动下标记为 $k(i)$ ，没有这样的活动时取 0，则有

$$c_i = \max\{c_{i-1}, c_{k(i)} + 1\}. \quad (9.2)$$

第一种方法要计算 $O(n^2)$ 个 c_{ij} ，每项至多枚举 n 个活动，因此需要 $O(n^3)$ 时间。第二种办法只需按活动下标依次计算，比枚举区间的办法更便捷，注意到结束时间已有序，每个 $k(i)$ 都可以二分查找，所以这个动态规划能在 $O(n \log n)$ 时间内完成。

然而我们希望得到更高效的算法，这就需求助于贪心算法。可以先试试下面非常自然的三种贪心策略，然而它们都不能保证得到最优解：

1. 先选开始最早的活动。图 9.1(a) 中，它会先选覆盖其他三个活动的长区间，最后只能举办一个活动，而选择上排则能举办三个。
2. 先选持续时间最短的活动。图 9.1(b) 中，中间短区间同时挡住左右两个区间，选它只能得到一个，左右两个却互相兼容。
3. 先选与其余活动冲突最少的活动。图 9.1(c) 中， x 只与 b, c 冲突，其余每个活动至少有三个冲突。选定 x 后，左边最多再选一个，右边也最多再选一个，总共至多三个，但最优解应当将 a, b, c, d 全部选中。

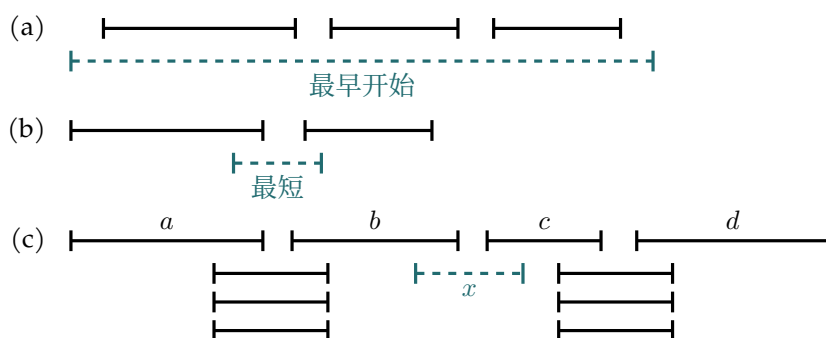


图 9.1 三种错误策略的反例。虚线标出该策略首先选择的活动的；同一横排的分离线段表示不同活动。

三个反例排除了三种看似合理的策略。但我们还有一个更有希望的策略，即每次选择结束最早且不冲突的活动：它结束得越早，后面可安排活动的时间就越充足，因此直观上能带来最优结果。接下来我们首先给出具体的算法描述，然后证明其正确性。选定活动 a_k 后，只需继续考虑在它结束后开始的活动的集合为 $S_k = \{a_i \in S \mid s_i \geq f_k\}$ 。因此，贪心算法先选结束最早的 a_1 ，再在 S_1 中作同样的选择，直到没有活动可选。下面给出了算法的伪代码。不难理解，算法的过程就是在活动按结束时间排好后，依次扫描，遇到与上一个已选活动兼容的活动就选入即可，只需 $O(n)$ 时间。若输入尚未排序，加上排序后总时间为 $O(n \log n)$ 。

算法 9.1 按结束时间选择活动

输入：活动 (i, s_i, f_i) , $0 \leq s_i < f_i$

输出：一个包含活动数最多的兼容活动列表

- 1: 按 f_i 递增排序
- 2: $A \leftarrow []$; $last \leftarrow 0$
- 3: **for** 排序后的每个活动 i **do**

```

4:   if  $s_i \geq last$  then
5:       将  $i$  追加到  $A$ ;  $last \leftarrow f_i$ 
6:   end if
7: end for
8: return  $A$ 

```

例如，下表活动已按结束时间排序，算法依次选中 a_1, a_4, a_8, a_{11} 。

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

接下来我们证明这一贪心算法的确能找到最优解。如果我们能证明：(1) a_1 确实在最优解中；(2) 原问题的最优解的确由活动 a_1 及子问题 S_1 的最优解构成，那么反复应用这一性质就能证明贪心算法的正确性。下面的两个定理正是要证明这两个性质。

定理 9.1 (活动选择问题贪心选择性质). 考虑任意非空子问题 S_k ，令 a_m 是 S_k 中结束时间最早的活动，则 a_m 在 S_k 的某个最大兼容活动子集中。

证明. 令 A_k 是 S_k 的一个最大兼容活动子集，且 a_j 是 A_k 中结束时间最早的活动。若 $a_j = a_m$ ，则已经证明 a_m 在 S_k 的某个最大兼容活动子集中。若 $a_j \neq a_m$ ，令集合 $A'_k = A_k - \{a_j\} \cup \{a_m\}$ ，即将 A_k 中的 a_j 替换为 a_m 。原方案中除 a_j 外的活动都在 f_j 之后或恰在 f_j 开始，而 $f_m \leq f_j$ ，所以换成 a_m 后，这些活动仍能照常进行。由于 $|A_k| = |A'_k|$ ，因此得出结论 A'_k 也是 S_k 的一个最大兼容活动子集，且它包含 a_m 。□

这一证明思想称为“交换参数法”，即可以假设存在一个最优选择，其中某个元素可能不在贪心选择中，然后通过交换贪心选择和最优选择的元素来构造一个不可能变差的解。这一思想在很多贪心算法的证明中都有应用。

定理 9.2 (活动选择问题最优子结构). 在活动选择问题中，用贪心策略选择 a_1 之后得到子问题 S_1 ，那么 a_1 和子问题 S_1 的最优解合并一定可以得到原问题的一个最优解。

证明. 反证法：假设 a_1 和子问题 S_1 的最优解 C_1 合并得到的解 C ，不是原问题的一个最优解。假设 C' 是原问题的一个最优解，则 $|C'| > |C|$ 。根据贪心选择性质可知，将 a_1 替换掉 C' 中的第一个元素得到 C'' 不会使得结果变差，即 $|C''| \geq |C'| > |C|$ ，那么将 C'' 除去 a_1 后，剩余的部分其实也是子问题 S_1 的一个解 C''_1 ，由于 $|C''| > |C|$ ，所以 $|C''_1| > |C_1|$ ，这和 C_1 是子问题 S_1 的最优解矛盾，因此原问题的最优解一定是 a_1 和子问题 S_1 的最优解合并得到的解。□

事实上，上述证明思路非常简单，就是反证法，如果综合起来不是最优解，那 C_1 也不可能是子问题最优解，因为能找到一个更优的解。事实上以前的动态规划问题的最优子结构也可以采用这样的证明方法，只是动态规划中都认为最优子结构性性质太显然所以没有特别强调。

对称地, 我们也可以从后往前选择最晚开始的活动作为贪心策略。看起来非常合理, 事实上是否正确呢? 答案是肯定的, 证明方法也和上面类似, 只是子问题要改为在已选活动开始前结束的活动集合, 不再赘述。

如果改变活动的收益或场地要求, 我们可以得到活动选择问题的两个常见的变体。

例题 9.3 (加权活动选择问题). 如果每个活动还有各自的收益, 目标改为最大化所选活动的总收益, 优先选最早结束的活动是否仍然正确?

解答. 不一定。例如两个活动分别为 $[0, 2)$ 、 $[1, 3)$, 收益分别为 1、100, 最早结束策略会错过收益更高的第二个活动。习题 9.1 要求用动态规划解决这一变体, 并恢复最优安排。□

例题 9.4 (教室分配问题). 假如所有活动都必须举办, 就需要准备多间教室。每间教室内的活动仍不能冲突, 那么至少需要多少间教室, 应该怎样分配?

解答. 解决思路非常简单直接: 将活动按开始时间排序, 再依次安排。安排一个活动时, 若有教室中的活动已经结束, 就复用这间教室; 若所有教室都还被占用, 就新开一间。

这样的算法为什么正确? 可以看下页图中的例子。图 (a) 展示了 10 个输入的活动, 事实上可以立刻断言: 至少也需要三个教室才能完成活动。原因很简单, 因为最多出现了三个活动同时进行的情况。而根据刚刚的方法, 不可能分配出超过 3 个教室的情况, 因为是出现冲突时才新开教室的, 如果开出了四个教室那就说明在某个时刻有四个活动同时进行, 这是不可能的。因此算法在这一场景下一定是正确的, 显然更进一步的推广到任意冲突的情况也是正确的。图 (b) 给出了一个可行的解。这里贪心算法正确性的证明没有像前面那样用贪心选择和最优子结构, 因为有更显然的教室数量上下界相等的性质可以更方便地说明。

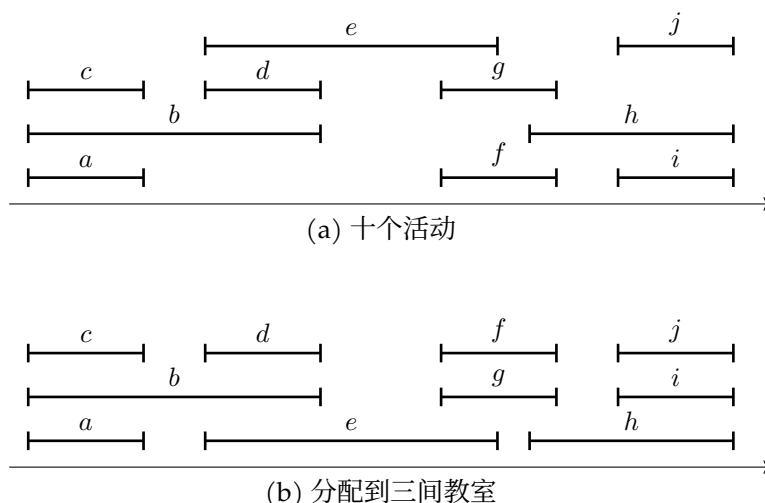


图 9.2 (a) 各活动占用的时间; (b) 使用三间教室的安排, 每一行代表一间教室。

实现时, 用最小堆保存各间教室最后一个活动的结束时间及教室编号, 堆顶对应最早空闲的教室。若堆顶结束时间不晚于新活动开始时间, 就将新活动安排在这里, 并更新结束时间, 否则只能增加一间教室。基于此, 时间复杂度为 $O(n \log n)$ 。□

9.2 调度问题

假设现在有 n 个任务，每个任务 i 都有执行时间 $l_i > 0$ 和权重 $w_i > 0$ 。假定只能按一定顺序依次执行这些任务，不能并行。所有任务从时刻 0 起都可以执行，一项任务开始后必须做完，才能执行下一项。

共有 $n!$ 种任务次序，我们希望找到其中加权完成时间之和最小的一种。记次序为 σ ，任务 i 的完成时间为 $C_i(\sigma)$ 。若从时刻 0 开始连续执行， $C_i(\sigma)$ 就等于排在 i 前面的任务耗时之和，再加上它自身的耗时 l_i 。我们的目标是最小化加权完成时间之和

$$T = \min_{\sigma} \sum_{i=1}^n w_i C_i(\sigma).$$

等待只会推迟任务的完成时间，所以最优方案应当从时刻 0 起连续执行任务。

举个例子，有三个任务， $l_1 = 1, l_2 = 2, l_3 = 3, w_1 = 3, w_2 = 2, w_3 = 1$ 。如果把第一个任务放在最前，第二个放在其次，第三个放在最后，那么三个任务的完成时间就是 1、3、6，因此加权时间和为 $3 \times 1 + 2 \times 3 + 1 \times 6 = 15$ 。如果读者检查全部的六种排序，这的确是最小的那一个。

我们希望按某个标准给任务排序，从而得到最优调度。一个任务有两个参数：执行时间 l_i 和权重 w_i 。我们先固定一个参数，看看另一个参数有什么影响：若所有任务耗时相同，应优先完成权重大的任务；若所有任务权重相同，应优先完成耗时短的任务。要同时照顾这两个因素，可以尝试以下两种排序方式：

1. 计算每个任务 i 的 $w_i - l_i$ ，按从大到小降序调度任务；
2. 计算每个任务 i 的 w_i/l_i ，按从大到小降序调度任务。

这两种方式都符合刚才的直觉，却可能排出不同的次序。例如，两个任务满足 $l_1 = 5, l_2 = 2, w_1 = 3, w_2 = 1$ 。按差值排序得到次序 2, 1，代价为 $1 \times 2 + 3 \times 7 = 23$ ；按比值排序得到 1, 2，代价为 $3 \times 5 + 1 \times 7 = 22$ 。这个例子排除了差值策略的正确性，下面我们证明比值策略对所有输入都正确，仍从贪心选择性质和最优子结构两方面证明。

定理 9.5 (调度问题的贪心选择性质). 令 i 是当前 w_i/l_i 最大的任务，则在当前问题下，一定存在将 i 排在首位的最优调度方式。

证明. 仍然使用“交换参数法”：假设有一个最优解 C ，如果它的第一个任务是 i ，结束证明。如果不是，考虑将 i 不断与前一个任务交换，直到换到第一个位置的过程。假设现在排在 i 前面一位的是 j ，则一定有

$$\frac{w_i}{l_i} \geq \frac{w_j}{l_j},$$

由于 $l_i, l_j > 0$ ，交叉相乘得到 $w_i l_j - w_j l_i \geq 0$ 。交换这两个相邻任务，不会改变它们前面的安排，也不会改变后面的任务何时开始，因为两项任务的总耗时仍为 $l_i + l_j$ 。所以只需比较 i, j 对目标值的贡献。设它们从时刻 t 开始执行，交换前的贡献为

$$t_1 = w_j(t + l_j) + w_i(t + l_j + l_i),$$

交换后为

$$t_2 = w_i(t + l_i) + w_j(t + l_i + l_j),$$

二者作差化简有 $t_1 - t_2 = w_i l_j - w_j l_i \geq 0$, 因此, 每次将 i 向前交换一位都不会增加总代价。从一个最优方案出发, 连续交换直到 i 排在首位, 得到的方案仍然最优。 $\square$

定理 9.6 (调度问题的最优子结构). 在调度问题 S 中, 用贪心策略首先选择了最大的 w_i/l_i 对应的任务 i 后, 剩下的子问题 S_1 (即在除 i 外的任务中寻找一个最小化加权完成时间之和的解) 的最优解 C_1 和 i 一起一定构成了原问题的一个最优解 C 。

证明. 反证法: 如果 C 不是最优解, 那么一定有一个最优解 C' , 它对应的加权完成时间之和 $T' < T$, 其中 T 是 C 对应的加权完成时间之和。根据贪心选择性质, 如果把 C' 中的 i 不断通过相邻交换换到第一个位置, 情况一定不会变差, 因此还是最优解, 将这一新的最优解记为 C'' 。于是 C'' 在选择了 i 之后, 剩下的选择实际上也是 S_1 的一个解, 由于 $T' < T$, 这表明 C'' 中对应的 S_1 的解必定比 C_1 更好, 但 C_1 是最优解, 因此得到矛盾。所以 C_1 和 i 一起一定构成了原问题的一个最优解 C 。 $\square$

总而言之, 我们只需要对每个 i 计算 w_i/l_i , 按从大到小排序后依次执行即可, 因此时间复杂度是 $O(n \log n)$ 。接下来我们来看这个问题的一些常见的变体。

例题 9.7 (最小化最大延迟). 设有 n 个任务, 任务 j 的执行时间为 $t_j > 0$, 截止日期为 d_j 。所有任务从时刻 0 起都可以执行, 由一台机器依次处理, 每项任务必须连续执行完毕。记任务 j 的完成时间为 C_j , 延迟时间为 $\max\{0, C_j - d_j\}$ 。请设计调度算法, 使所有任务的最大延迟最小, 即最小化 $L = \max_j \max\{0, C_j - d_j\}$ 。

解答. 沿用前面按某个数值排序的思路, 可以先尝试如下两种贪心策略:

1. 优先执行耗时短的任务。例如两项任务的 (t_j, d_j) 分别为 $(1, 100)$ 和 $(10, 10)$, 先做短任务会使第二项任务延迟 1; 反过来执行, 两项任务却都能按时完成。因此只看耗时并不可靠。
2. 优先执行 $d_j - t_j$ 较小的任务。这个差表示任务要按时完成, 最晚可以在什么时候开始, 看起来越小就越紧迫。但考虑 $(t_j, d_j) = (1, 2)$ 和 $(10, 10)$: 按此规则先做第二项任务, 第一项任务到时刻 11 才完成, 延迟为 9; 先做第一项任务, 则两项的延迟分别为 0 和 1, 最大延迟只有 1。

这两个反例说明, 只按耗时排序或按 $d_j - t_j$ 排序都不可靠。事实上, 我们可以直接按截止日期 d_j 从早到晚安排任务。虽然这个次序没有用到耗时, 但可以证明, 把截止日期较早的任务换到前面不会增大最大延迟: 若次序不是按截止日期递增排列, 就存在相邻任务 i, j , 其中 i 在前而 $d_i > d_j$ 。设它们开始于时刻 t , 交换后其他任务的完成时间不变, 任务 j 提前完成; 任务 i 虽然推迟, 但其新的完成时间恰好等于 j 原来的完成时间。因此

$$\max\{0, C_i^{\text{new}} - d_i\} \leq \max\{0, C_j^{\text{old}} - d_j\}.$$

两个任务交换后的延迟都不超过交换前的最大延迟, 所以整体目标不会变差。因此从任意的最优解出发, 按上述规则交换任务, 可以得到一个按截止日期递增排列且结果不变差的解, 因此按截止日期从小到大排序是最优的。

上述论证无需进一步证明最优子结构性质。当然我们也可以遵循此前证明的模板, 不过剩余任务的起始时间应当是已经消耗的时间 t , 或者把每个剩余任务的截止日期改为 $d_j - t$ 。 $\square$

例题 9.8 (最小化延时惩罚). 假设每项任务都只需一个单位时间, 任务 j 若没有在截止日期 d_j 前或恰在该时刻完成, 就产生惩罚 w_j 。这里 d_j 为正整数, $w_j \geq 0$, 所有任务都从时刻 0 起可以执行, 仍由一台机器连续处理。目标是安排执行顺序, 使总惩罚最小。

关于这一问题, 我们将在第 9.3.3 节用拟阵给出一个贪心算法。

例题 9.9 (完成工时最小化). 假设有 n 项作业, 每个作业 j 具有已知的长度 l_j , 不同于前面的问题都只有一台机器顺序执行所有工作, 这里有 m 台相同的机器可以同时处理作业。希望找到一个调度方案, 使得最后一个完成的作业尽早结束。

举个简单的例子, 假如有两台一样的机器, 作业长度分别为 $[1, 2, 2, 3]$, 那么可以将作业分配为 $[1, 2], [2, 3]$, 这一分配的完成工时为 5。如果将作业分配为 $[1, 3], [2, 2]$, 那么完成工时为 4, 恰好达到总工作量除以机器数的下界 $(1 + 2 + 2 + 3)/2 = 4$ 。

这一问题也有容易实现的贪心策略, 例如每次把下一项作业交给当前最早空闲的机器, 但它未必得到最优解。近似算法一讲会分析这类安排与最优结果之间可能相差多少。

9.3 贪心算法的范式

9.3.1 贪心算法的解决步骤

贪心算法通过做出一系列选择来求出问题的最优解。在每个决策点, 它做出在当时看来最佳的选择。这种启发式策略并不保证总能找到最优解, 但对有些问题确实有效, 如活动选择问题。一般地, 可以按如下步骤设计贪心算法:

1. 将最优化问题转化为这样的形式: 对其做出一次选择后, 只剩下一个子问题需要求解;
2. 证明做出贪心选择后, 原问题总是存在一个包含该贪心选择的最优解, 即贪心选择总是安全的;
3. 证明做出贪心选择后, 剩余的子问题满足性质: 其最优解与贪心选择组合即可得到原问题的最优解, 这样就得到了最优子结构。

可以代入活动选择问题来看这一范式的应用, 回忆对子问题 S_k 的定义以及正确性的证明, 上述思路是非常清晰的。下面的问题是如何证明一个贪心算法是否能求解一个最优化问题呢? 并没有适合所有情况的方法, 例如之前涉及的“交换参数法”并不是唯一的证明贪心选择正确性的方法, 在例 9.4 中已经见到了对于特定问题更简单的证明方式。如果读者回顾 Dijkstra 算法, 就会发现在那里是利用了数学归纳法来证明算法的正确性的, 而最小生成树的两种算法正确性证明则涉及了图问题的一些更复杂的特性。当然这里学到的“交换参数法”是对于很大一部分问题比较直观的一种证明手段, 但贪心这一思想实在是应用太广泛, 所以无法保证这种证明方法对所有问题都适用。

值得一提的是贪心算法与动态规划之间的相似性与差别。两者在最优子结构（一个问题的最优解包含其子问题的最优解）上具有相似性, 但动态规划在做出每一步选择的时候通常依赖于子问题的解, 而贪心算法则不依赖于子问题的解。

当然还有一个很困难的问题就是如何找到一个合理的贪心思路: 这的确有些困难, 有时候你可以去考虑一些极端情况, 从中会发现具有哪些性质的解是最想要的（例如调度问题用这一思想就很容易想到可能的解法）。在多参数（例如调度问题有权重、完成时间、截止时间等参数）的情况下, 也

可以固定某些参数看另一些参数的变化等，这些小技巧都能帮助对问题的基本结构有一个观察。最后在证明时也是不应完全被套路束缚，有时候也应当从一些直观中找到灵感，特别是有的时候最优子结构其实非常显然，例如前面的两个问题就是如此，所以直观的感受结合将直观转化为严谨的叙述的过程是很重要的。当然还有一种统一的理论可以帮助我们为一类问题提供一个通用的贪心算法设计思路，这就是拟阵理论。

9.3.2 拟阵

假如要从一组价值非负的物品中选出总价值最大的一部分，最容易想到的办法就是按价值从高到低选择。如果只限制最多选 k 件，这样做当然正确；但若物品之间还有冲突，结果就未必了。例如，三个活动的时间分别为 $[0, 4), [0, 2), [2, 4)$ ，收益分别为 3、2、2。先选收益最大的活动只能得到 3，而选后两个可以得到 4。

那么，选择时的约束满足什么条件，才能保证这种贪心方法正确？拟阵 (matroid) 给出了一组充分条件。下面用 E 表示全部元素，用 $\mathcal{I}$ 收集所有允许选出的集合。

定义 9.10 (拟阵). 设 E 是有限集合， $\mathcal{I}$ 是由 E 的若干子集组成的非空集合族。如果满足以下两条性质，就称 $(E, \mathcal{I})$ 为拟阵， $\mathcal{I}$ 中的集合称为独立集。

1. **遗传性**。从一个独立集中删去一些元素，剩下的集合仍然独立，即

$$B \in \mathcal{I}, A \subseteq B \implies A \in \mathcal{I}.$$

2. **增广性质**。对于两个独立集，如果其中一个的元素较少，就能从另一个集合中取出一个它没有的元素，加入后仍然独立，即

$$A, B \in \mathcal{I}, |A| < |B| \implies \exists e \in B \setminus A, A \cup \{e\} \in \mathcal{I}.$$

由于 $\mathcal{I}$ 非空，而空集是其中任意集合的子集，遗传性还保证了 $\emptyset \in \mathcal{I}$ 。增广性质也常称交换性质。不难看出，上述定义将矩阵中行/列向量的线性无关性质抽象为上述遗传性和增广性质，这也是拟阵名称的由来。

例如，最多选 k 件物品时， $\mathcal{I} = \{A \subseteq E : |A| \leq k\}$ ，其中 k 为非负整数。删去物品后数量仍不超过 k ，所以满足遗传性。若 A, B 都满足数量限制，且 $|A| < |B|$ ，从 $B \setminus A$ 中任取一件加入 A 后，数量仍不超过 $|B| \leq k$ ，所以也满足增广性质。

最小生成树中的选边约束也构成拟阵。在一个无向图中，令 E 为全部边， $\mathcal{I}$ 为所有不含环的边集，就得到了图拟阵。删去边不会产生环，遗传性成立。为证明增广性质，取同一图中两个不含环的边集 A, B ，满足 $|A| < |B|$ 。只要 B 中有一条边连接了 A 的两个不同连通分支，就可以把这条边加入 A ，而不会产生环。假设找不到这样的边，那么 B 中每条边的两端都在 A 的同一连通分支内。对于 A 的一个含 v 个顶点的分支， A 有 $v - 1$ 条边，而 B 在这些顶点之间至多也只有 $v - 1$ 条边，否则就会成环。把各分支的边数相加，得到 $|B| \leq |A|$ ，与已知条件矛盾。因此一定能找到所需的边，增广性质成立。

再看开头的三个活动。删去活动当然不会产生时间冲突，但 $A = \{[0, 4)\}$ 和 $B = \{[0, 2), [2, 4)\}$ 虽然各自兼容， B 中却没有任何活动能加入 A 。它们不满足增广性质，因而活动选择的可行集合一般不构成拟阵。

定理 9.11 (拟阵的加权贪心). 给拟阵的每个元素赋予非负权重。从空集开始, 按权重从大到小检查元素, 只要加入后仍独立就选入, 所得独立集的总权重最大。

证明. 将元素按权重排为 $e_1, \dots, e_n$, 其中权重 $w_1 \geq \dots \geq w_n \geq 0$ 。记 $E_i = \{e_1, \dots, e_i\}$, 贪心算法最后得到的集合为 G 。任取一个独立集 O , 我们先比较 G 和 O 在前 i 个元素中各选了多少个。假设 O 选得更多, 即 $|O \cap E_i| > |G \cap E_i|$ 。这两个交集都是独立集, 由增广性质, 可以从 $O \cap E_i$ 中找到一个不属于 G 的元素 e , 使 $(G \cap E_i) \cup \{e\}$ 仍然独立。也就是说, 贪心选完前 i 个元素后, 本来还可以再选入 e , 因此产生了矛盾, 所以对每个 i 都有 $|G \cap E_i| \geq |O \cap E_i|$ 。

还需要把数量上的比较变成权重上的比较。令 $w_{n+1} = 0$, 把第 j 个元素的权重写成

$$w_j = (w_j - w_{j+1}) + \dots + (w_n - w_{n+1}).$$

其中, 差值 $w_i - w_{i+1}$ 出现在前 i 个元素的权重中。因此, 对一个集合 A 中元素的权重求和时, 这个差值恰好计算 $|A \cap E_i|$ 次, 于是

$$w(A) = \sum_{i=1}^n (w_i - w_{i+1}) |A \cap E_i|.$$

由于每个差值都非负, 且 G 在每个前缀中选入的元素数都不少于 O , 代入上式就得到 $w(G) \geq w(O)$, 命题得证。□

这个结论能解释 Kruskal 算法为什么正确。对于有 n 个顶点的连通图, 取一个大于所有边长的常数 K , 把边长 $c(e)$ 改成权重 $w(e) = K - c(e) > 0$ 。按新权重从大到小选边, 就是按原边长从小到大选边。由于新权重都为正, 最优森林必须一直加边, 直到成为生成树, 否则还能加边提高总权重。由此可见, 拟阵贪心在这里恰好就是 Kruskal 算法。

总结而言, 如果我们能证明我们希望构造的集合符合独立集的定义, 那么就可以应用拟阵贪心算法来解决相关问题。下面我们利用这一思想来解决单位时长任务的最小惩罚调度问题。

9.3.3 用拟阵解决单位时长任务的最小惩罚调度

回到例 9.8, 每项任务都需要一个单位时间, 逾期后要支付相应惩罚。如果所有任务都逾期, 总惩罚就是 $\sum_i w_i$; 每让一项任务按时完成, 就能从中省下它的惩罚 w_i 。因此, 我们可以先选择一组能够全部按时完成任务, 使它们的惩罚之和最大, 再安排执行顺序, 这就等价于最小化超时集合的总惩罚。

令 E 为全部任务的集合, 将能够安排在各自截止日期前完成的任务集合组成集合族 $\mathcal{I}$ 。我们要证明 $(E, \mathcal{I})$ 是拟阵, 这样就能把任务 i 的惩罚 w_i 作为权重, 用拟阵贪心选出总权重最大的独立集。

为证明这两条拟阵性质, 先找出判断 $A \in \mathcal{I}$ 的条件。任取任务集合 $A \subseteq E$, 记 $N_t(A)$ 为其中截止日期不晚于 t (其中 t 为非负整数) 的任务数, 即 $N_t(A) = |\{i \in A : d_i \leq t\}|$ 。在时刻 t 之前, 机器最多完成 t 项任务, 所以必须有 $N_t(A) \leq t$ 。全部 n 项任务连续执行也只需 n 个单位时间, 故可将大于 n 的截止日期改为 n , 只检查 $t = 0, 1, \dots, n$ 。

事实上, 这些条件也足以保证 A 中的任务全部按时完成。将 A 中任务按截止日期递增排列, 从时刻 0 开始连续执行。假设第 j 项任务逾期了, 其截止日期为 $t < j$, 那么前 j 项任务都要求在时刻 t 前完成, 意味着 $N_t(A) \geq j > t$, 与条件矛盾。因此, $A \in \mathcal{I}$ 当且仅当

$$N_t(A) \leq t \quad (t = 0, 1, \dots, n),$$

而按截止日期递增执行就是一种可行安排。

现在证明 $(E, \mathcal{I})$ 满足拟阵定义。空集属于 $\mathcal{I}$ ，所以 $\mathcal{I}$ 非空。对于任意 $A \in \mathcal{I}$ ，删去其中一些任务后，其余任务仍能按时完成，因此遗传性成立。为证明增广性质，取 $A, B \in \mathcal{I}$ ，满足 $|A| < |B|$ ，我们要从 $B \setminus A$ 中找一项任务加入 A ，使新集合仍属于 $\mathcal{I}$ 。

考虑哪些时刻之前必须完成的任务数已经达到上限。令 k 为最后一个满足 $N_k(A) = k$ 的时刻，这样的时刻至少有 0。由于 $|A| < |B| \leq n$ ，时刻 n 不满足等式，所以 $k < n$ 。对于每个 $t > k$ ，都有 $N_t(A) < t$ ，还可以多容纳一项任务。因此，只要找到一项截止日期大于 k 的新任务，就能加入 A 。

这样的任务一定能在 $B \setminus A$ 中找到。 A 恰有 k 项任务的截止日期不晚于 k ，而 B 至多有 k 项，因此 $|B| - N_k(B) > |A| - k$ 。左边是 B 中截止日期大于 k 的任务数，右边是 A 中这类任务的数量。前者更多，所以至少有一项属于 B 而不属于 A 。将它加入 A 后， $t \leq k$ 的计数不变， $t > k$ 的计数至多增加 1，仍不超过 t 。增广性质得证。

于是可以按惩罚从大到小选任务，每次只检查加入后是否仍能全部按时完成。设新任务的截止日期为 d ，加入它会使 $N_d, N_{d+1}, \dots, N_n$ 各增加 1，其余计数不变。如果这些计数原来都小于对应的 t ，就选入该任务；只要有一项已经等于 t ，就跳过它。如果用 A 保存选入的任务，用 R 保存跳过的任务。根据拟阵贪心性质， A 的总惩罚最大，从而超时集合的最小总惩罚恰为 $\sum_{i \in R} w_i$ 。

总结算法的时间复杂度。按惩罚排序的时间复杂度为 $O(n \log n)$ ，构造集合 A 和 R 的过程中，对每个任务都需要检查 $O(n)$ 个条件是否成立，共有 n 个任务需要检查，所以总时间复杂度为 $O(n^2)$ 。

最后我们来看一个简单的例子：假设七个任务的数据为

i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

按惩罚递减检查时，前四项任务都可以选入。此时，要求在时刻 4 前完成的任务已有四项， $N_4 = 4$ 。任务 5、6 也都要求在时刻 4 前完成，无论再加入哪一项，都会使 N_4 超过 4，所以两项都要跳过。任务 7 的截止日期为 6，可以接在已选任务后按时完成，因而选入，总惩罚为 $30 + 20 = 50$ 。

9.4 哈夫曼编码

假如要把一段文本保存成二进制串，最简单的办法是给每个字符分配一个等长的编码。但文本中的字符往往并非一样常见，如果让常见字符的编码短一些，即使少见字符的编码长一些，也可能减少总长度。当然，收到这串二进制后，还必须能反过来恢复出原文。我们先介绍能够直接逐个读出字符的前缀码，再用哈夫曼（Huffman）算法求出平均码长最短的前缀码。

9.4.1 前缀码与编码树

给每个字符分配的 0-1 串称为它的码字。如果 a 编为 0， b 编为 01， c 编为 1，收到 01 时就无法判断原文是 b 还是 ac 。为了避免这种歧义，可以要求任意一个字符的码字都不是另一个字符码字的前缀，这样的编码称为前缀码。从左向右读取时，一旦读到了一个完整码字，就能确定一个字符，因为它不会再与后面的比特组成另一个码字。

前缀码可以用二叉树表示。从根出发，遇到码字中的 0 就向左走，遇到 1 就向右走，并将字符放在路径终点。图 9.3 给出两棵前缀码的编码树。一个值得注意的点是，前缀码树中的内部结点可能只

有一个孩子，例如码字为 00、01、10 时，对应路径 1 的结点就只有一个孩子。但我们可以把这样的结点删去，让它的孩子接替其位置，其下每个字符的码字都会缩短一位，字符之间仍满足前缀条件。因此，当字符至少有两种时，最优编码树的每个内部结点都有两个孩子。



图 9.3 同一字符串的两种前缀编码。字符都在叶子上，从根沿边标签得到码字。

考虑字符串 **aaaxuaxz**，其中字符的频数为 $f_a = 4, f_u = 1, f_x = 2, f_z = 1$ 。定长编码 $a = 00, u = 01, x = 10, z = 11$ 需要 $2 \times 8 = 16$ 位；若改用 $a = 0, u = 110, x = 10, z = 111$ ，则只需要 $4 + 3 + 4 + 3 = 14$ 位。更一般的，记字符 c 在编码树中的深度为 $d_T(c)$ ，则编码的总长度为

$$B(T) = \sum_{c \in C} f_c d_T(c).$$

文本共有 $F = \sum_c f_c$ 个字符，所以平均码长为 $B(T)/F$ 。

9.4.2 信息熵与平均码长

前面的例子中，改变编码方式可能减少编码所需的比特数。一个自然的问题是，平均每个字符至少需要多少位？事实上，这个下界由字符出现的概率决定，可以用香农定义的“信息熵”表示。

定义 9.12 (信息熵). 对于一个离散随机变量 X ，其信息熵定义为

$$H(X) = - \sum_x p(x) \log_2 p(x) \quad (9.3)$$

其中 $p(x)$ 是 X 取值为 x 的概率，约定 $0 \log_2 0 = 0$ 。

对数以 2 为底时，熵的单位是比特。若 X 表示随机出现的字符，则任何二进制前缀码的平均码长都至少为 $H(X)$ 。这里给出两个能恰好达到这个下界的例子，证明将在补充材料中给出。

例题 9.13. 一个随机变量 X 有 32 种等概率的取值。如果要用二进制编码告诉别人它的取值，平均至少需要多少位？能否达到这个下界？

解答. 由于 $2^5 = 32$ ，给每种取值分配一个不同的 5 位码字，就能区分全部结果。而信息熵为

$$H(X) = - \sum_{i=1}^{32} \frac{1}{32} \log_2 \frac{1}{32} = 5 \quad (9.4)$$

所以平均码长至少为 5 位，定长编码恰好达到这个下界。 □

例题 9.14. 假定有 8 匹马参加一场比赛，它们的获胜概率分别为 $\left(\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}, \frac{1}{64}\right)$ 。如果要把比赛的获胜者告诉别人，怎样编码才能使平均码长最短？

解答. 以获胜者为随机变量 X ，其信息熵为

$$H(X) = - \sum_{i=1}^8 p_i \log_2 p_i = 2 \quad (9.5)$$

假定比赛结束后，我们要把哪匹马获胜的结果告诉别人，其中一个策略是发送胜出的马的编号，这样对于任何一匹马都需要 3 个比特。但由于概率不是均等的，明智的方法是对概率大的马用更短的编码，对概率小的马用更长的编码。例如使用以下编码：0, 10, 110, 1110, 111100, 111101, 111110, 111111，按各匹马获胜的概率加权，传递一次比赛结果平均需要 2 个比特，恰好达到熵给出的下界。□

在这两个例子中，每个码字的长度恰好都是 $-\log_2 p_i$ ，所以平均码长等于熵。但码长必须是整数，因此通常无法这样分配。例如，两个字符的概率为 0.9, 0.1 时，最短的前缀码是 0、1，平均码长为 1，而熵只有约 0.469。第 9.6 节将证明，对至少两个正概率字符，哈夫曼编码的平均码长 L 满足 $H(X) \leq L < H(X) + 1$ 。

有很多人都尝试从信息论角度研究最优的前缀编码。1951 年，哈夫曼和他在 MIT 信息论的同学需要选择是完成学期报告还是期末考试。导师 Robert M. Fano 给他们的学期报告的题目是，寻找最有效的二进制编码。由于无法证明哪个已有编码是最有效的，哈夫曼放弃对已有编码的研究，转向新的探索，最终发现了基于有序频率二叉树编码的想法，并很快证明了这个方法是最有效的。由于这个算法，学生终于青出于蓝，超过了他那曾经和信息论创立者香农共同研究过类似编码的导师。事实上哈夫曼的构造思想在编码理论中算得上独树一帜的，因为这一贪心算法的构造的确与当时主流的想法完全不同，接下来我们就将详细介绍哈夫曼编码的构造过程。

9.4.3 哈夫曼编码的构造

前面提到，出现次数少的字符可以使用较长的码字，在编码树中也就可以放得更深。哈夫曼算法从这里入手，先把频数最小的两个字符放在一对兄弟叶结点上，再将它们合起来看作一个新字符，频数取两者之和。这样，待处理的字符就少了一个，然后继续做同样的贪心选择。

具体构树时，开始的每个字符都是一棵单结点树。每次选出权重最小的两棵树，为它们建立一个共同的父结点，就合成了一棵新树。新树再次参与选择时，需要把其中所有字符作为一个整体，所以它的权重取两棵树的权重之和，也就是其中全部字符的频数之和。不断合并，直到所有字符都在同一棵树中，就得到了编码树。下面给出一个例子，说明如何构造哈夫曼编码。

例题 9.15. 取字符 $a, e, i, s, t, \text{SP}, \text{NL}$ ，其中 SP 为空格、NL 为换行，频数依次为 10, 15, 12, 3, 4, 13, 1。请构造一组哈夫曼编码，并计算编码后的总长度和平均码长。

解答. 先合并频数为 1 的换行符和频数为 3 的 s ，得到权重为 4 的树。此时，这棵树与频数为 4 的 t 最小，再将它们合并为权重为 8 的树。接下来把这棵树与频数为 10 的 a 合并，随后继续选择当前最小的两项，最终得到图 9.4。沿根到各叶子的路径读出 0、1，可以得到下表。

字符	<i>a</i>	<i>e</i>	<i>i</i>	<i>s</i>	<i>t</i>	SP	NL
码字	111	10	00	11011	1100	01	11010
频数	10	15	12	3	4	13	1

因此总长度 $3 \times 10 + 2 \times 15 + 2 \times 12 + 5 \times 3 + 4 \times 4 + 2 \times 13 + 5 \times 1 = 146$ 位。文本共有 58 个字符，所以平均码长为 $146/58$ 。□

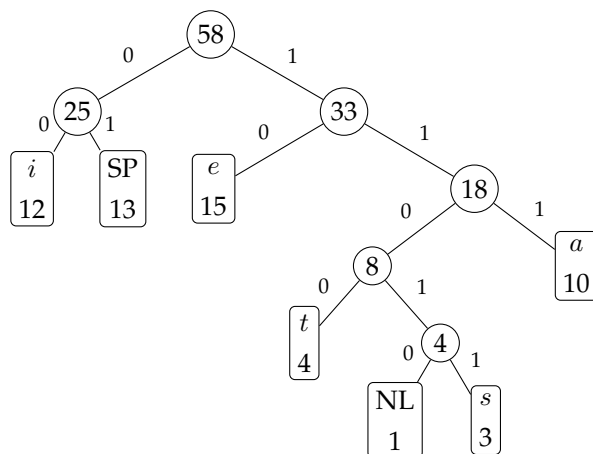


图 9.4 七个字符的 Huffman 树。叶结点标出字符及其频数，内部结点标出其下所有字符的频数之和。

由于算法每一步都要取出两个最小权重，并放回一个新权重，这正适合用最小堆实现。

算法 9.2 构造哈夫曼编码树

输入：字符及正频数 (c_i, f_i)

输出：哈夫曼编码树根

- 1: 按字符编号顺序建立叶子，权重为 f_i ，序号为 $1..m$
- 2: 用这些结点建立最小堆 H ，按权重、序号进行字典序比较
- 3: **if** $H = \emptyset$ **then**
- 4: **return** NIL
- 5: **end if**
- 6: **while** $|H| > 1$ **do**
- 7: $x \leftarrow H.\text{EXTRACTMIN}$; $y \leftarrow H.\text{EXTRACTMIN}$
- 8: 创建内部结点 z , $z.\text{left} \leftarrow x$, $z.\text{right} \leftarrow y$
- 9: $z.\text{weight} \leftarrow x.\text{weight} + y.\text{weight}$; 赋予 z 下一个序号
- 10: $H.\text{INSERT}(z)$
- 11: **end while**
- 12: **return** $H.\text{EXTRACTMIN}$

根据上述过程可知，哈夫曼编码并不唯一，因为可能遇到最小权重相同的情况，或者可以交换左右子树的位置。但要注意的是，这些差异并不会改变最优的总码长。下面分析时间复杂度：设共有 $m \geq 1$ 种字符，则最初有 m 棵树。每次合并减少一棵，要合成一棵就需要 $m - 1$ 次。每次新建一个

内部结点, 加上最初的 m 个叶子, 共有 $2m - 1$ 个结点。用二叉堆实现时, 最初建堆需要 $O(m)$ 时间, 每次合并要做两次删除最小值和一次插入, 各需 $O(\log m)$ 时间, 因此构树总时间为 $O(m \log m)$ 。

9.4.4 哈夫曼编码的最优性证明

接下来证明哈夫曼编码的最优性。哈夫曼算法第一次合并时, 将频数最小的两个字符放在同一个父结点下面。此后无论怎样合并, 这两个字符都会在未来的编码树中互为兄弟。因此, 首先要证明存在一棵最优编码树, 也让这两个字符互为兄弟, 这样才能保证第一次合并没有排除所有最优解。在此基础上, 再证明将它们合并后, 只要求出剩余问题的最优编码, 就能还原出原问题的最优编码。

引理 9.16 (贪心选择性质). 设字母表 C 至少含两个字符, 各字符的频数 f_c 均为正。令 x, y 为频数最小的两个字符, 则存在一棵最优前缀码树, 使 x, y 是兄弟叶结点, 即它们的码字长度相同, 且只有最后一位不同。

证明. 取一棵最优树 T , 找到其中一对最深的叶子 (如图 9.5 中 a, b)。将任意结点 n 的深度记为 $d(n)$ 。我们把 x, y 换到这两个位置。先考虑 x 不在这两个位置上的情况, 将它与其中的字符 a 交换。交换后, x 的码长增加 $d(a) - d(x)$, a 的码长则减少同样多。由于 x 的频数不大于 a , 增加的代价不会超过节省的代价。如果 x 本来就在这两个位置之一, 就保留它, 接着将 y 换到另一个位置。同样的分析表明, 交换后总代价不会增加。于是我们得到一棵最优树, 使 x, y 互为兄弟。□

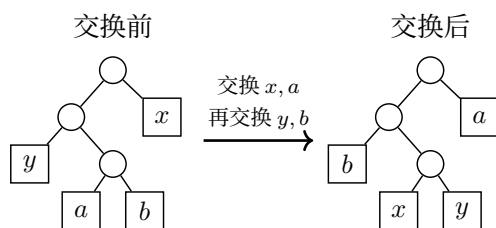


图 9.5 字符交换的树形示意。把 x, y 移到最深的一对兄弟叶子上, 树形不变。

上面的引理保证, 至少有一棵最优树将 x, y 放在同一个父结点下面。把这两个叶子及其父结点收缩为一个叶子 z , 就得到一个少了一个字符的编码问题, 其中 $f_z = f_x + f_y$ 。下面证明, 只要求出这个较小问题的最优编码, 再把 z 展开为以 x, y 为孩子的内部结点, 所得编码就是原问题的最优解。

引理 9.17 (最优子结构性质). 设 x, y 是字母表 C 中频数最小的两个字符, 各字符频数均为正。将它们合并为频数 $f_z = f_x + f_y$ 的新字符 z , 得到字母表 C' 。取 C' 的任意一棵最优前缀码树 T' , 将其中的叶结点 z 改为内部结点, 并以 x, y 为它的两个孩子, 所得树 T 就是 C 的最优前缀码树。

和此前活动选择问题一样, 使用反证法可以非常轻松地证明, 故不再赘述。这样, 原问题就归结为少一个字符的同类问题, 按字符数归纳, 便证明了整个哈夫曼算法的正确性。

9.5 匹配问题

匹配问题是算法博弈论中的经典问题, 本节以房屋交换和男女配对为例, 讨论具体的匹配问题与算法。

9.5.1 房屋交换问题

假如有 n 个人，每人拥有一间房屋，但未必最喜欢自己的房子。如果两个人恰好都喜欢对方的房子，他们就可以交换。当然也可能需要三人合作，例如参与人 1 想要参与人 2 的房子、参与人 2 想要参与人 3 的房子、参与人 3 又想要参与人 1 的房子。形式化而言，将参与人和房屋分别编号为 $1, \dots, n$ ，房屋 i 初始属于参与人 i 。每个人按喜好给全部房屋排序，假定没有并列。最终每人得到一间房屋，每间房屋也只分给一人。我们要求分配满足以下两个条件：

1. 每个人得到的房屋至少与原住房一样好，称为个体理性。
2. 在这个结果上，无法重新分配房屋，使所有人都不变差，同时至少一人变好，称为帕累托有效。

要找到可以交换的一群人，可以先让每个人指出自己最喜欢的房屋。把人画成顶点，如果参与人 i 最想要参与人 j 的房屋，就画一条 $i \rightarrow j$ 的箭头，想保留自己房屋的人则指向自己。任取一人，沿箭头一直走下去，注意总人数有限，因此一定会遇到已经经过的人，从而找到一个环。环上的人可以同时交换。例如 $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ 时，把房屋 2 分给参与人 1、房屋 3 分给参与人 2、房屋 1 分给参与人 3，三人便都拿到自己的首选。这些人得到房屋后便退出。对于其余人，再让每个人指出剩余房屋中最喜欢的一间，按同样的方法找环并交换，直到全部分配完毕。这就是**首选交易环**（Top Trading Cycles, TTC）算法。一个人指向自己时，也构成一个环，表示他保留自己的房屋后退出。由于每轮至少有一人得到房屋，最多 n 轮就能分配完毕。

例如，四个人的偏好如下，其中房屋编号越靠左，表示越喜欢：

参与人	偏好顺序	参与人	偏好顺序
1	2 > 1 > 3 > 4	2	1 > 2 > 3 > 4
3	2 > 4 > 3 > 1	4	3 > 4 > 1 > 2

第一轮中，参与人 1 和参与人 2 互相指向对方，交换房屋后退出。参与人 3 也想要房屋 2，但不在这个环上，暂时无法得到它。剩下房屋 3、4 后，参与人 3 最喜欢房屋 4，参与人 4 最喜欢房屋 3，两人再交换。最终，四人按编号顺序得到的房屋依次为 2, 1, 4, 3。

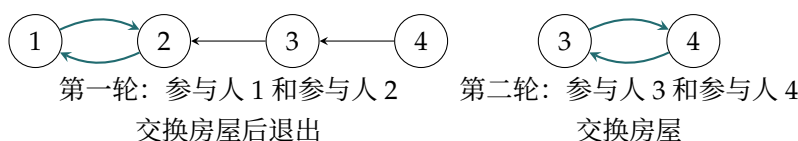


图 9.6 TTC 算法示例。

实现时，为每个人保存一个指针，表示在偏好表中查找到了哪里。每轮从这个位置继续向后查找，跳过已经分配的房屋，遇到的第一间剩余房屋就是他本轮的首选。这一过程中每个人的指针只向后移动，最多移动 $n - 1$ 次，因此所有指针移动的总时间为 $O(n^2)$ 。然后绘制每个人的箭头并寻找环，复杂度是 $O(n)$ 的，所以总时间为 $O(n^2)$ 。

定理 9.18 (TTC 的分配性质). 在没有并列偏好的条件下，TTC 所得分配满足个体理性和帕累托有效。

证明. 首先, 一个人退出时, 自己的房屋仍可选择, 而算法让他拿到剩余房屋中最喜欢的一间。因此, 得到的房屋不会比原住房差, 个体理性成立。

下面证明帕累托有效。假设重新分配后没有人变差, 那么第一轮退出的人必须保留算法分给他们的房屋, 因为他们已经得到全部房屋中的首选, 改分任何其他房屋都会更差。这些房屋的归属既然不变, 第二轮退出的人能够选择的仍是当时剩下的房屋, 而算法也已经给了他们其中最喜欢的一间。因此, 第二轮的分配同样不能改变。按此推理, 每一轮的分配都必须保持不变, 也就不可能让任何人变好。 $\square$

房屋交换中的环也能帮助我们理解肾脏配对捐献。假设患者甲的亲友愿意为他捐肾, 但医学检查发现两人不相容, 患者乙与自己的捐献者也有同样的困难。如果甲的捐献者适合向乙捐献, 乙的捐献者又适合向甲捐献, 就可以交叉安排两次捐献, 让两位患者都获得相容的肾脏。两组之间无法直接交换时, 也可能由三组或更多组组成一个交换环。¹

把每组患者与其捐献者看作一个顶点, 如果第 i 组的捐献者能向第 j 组的患者捐献, 就画边 $i \rightarrow j$ 。一个有向环便给出一组可以相互帮助的捐献安排, 这与房屋交换中沿环分配的思路相通。不过, 这里的边表示医学上的相容性, 每个顶点可能有多条出边。实际安排还要考虑手术协调和交换环长度(长度过长会导致同时手术的场次过多)等限制, 通常希望选出若干个互不重叠的环, 让尽量多的患者接受移植。因此, 本节 TTC 的找环思想可以帮助理解这项应用, 实际求解时还需要根据这些条件设计专门的算法。²

9.5.2 稳定匹配问题

房屋交换中, 房屋本身没有偏好, 只需考虑人想住哪一间。如果要给两组人配对, 则需要同时考虑双方的选择。假设有 n 个男生和 n 个女生, 每个人都按喜好给全部异性排序, 没有并列。我们希望将他们一一配对。假如某个未配成一对的男生和女生都喜欢对方胜过喜欢各自当前的对象, 那么这两个人就都有理由离开原来的配对。称这样的一对人为阻塞对, 没有阻塞对的配对方案称为稳定匹配, 我们希望找到一个稳定匹配。

一种自然的尝试是让男生先向自己最喜欢的女生申请。但几位男生可能喜欢同一个女生, 女生无法同时接受所有人, 而且后面还可能收到更喜欢的男生的申请。为了解决这一问题, Gale-Shapley 算法先不确定最终配对, 让女生暂时保留到目前为止最喜欢的申请者。具体来说, 每次找一位尚未被接受的男生, 让他向还没有申请过的女生中最喜欢的一位申请。如果女生还没有对象, 就暂时接受他; 如果已经有了对象, 就比较两人, 保留自己更喜欢的那一位。被拒绝的男生继续向偏好表中的下一位申请。直到所有男生都被暂时接受, 即可确定最终配对。这就是延迟接受算法。每个男生对同一女生只申请一次, 总共至多有 n^2 次申请, 因此总时间为 $O(n^2)$ 。

例如, 有三位男生 1, 2, 3 和三位女生 a, b, c , 双方的偏好如下。

男生	对女生的偏好	女生	对男生的偏好
1	$a \succ b \succ c$	a	$2 \succ 3 \succ 1$
2	$b \succ a \succ c$	b	$1 \succ 3 \succ 2$
3	$a \succ b \succ c$	c	$3 \succ 1 \succ 2$

¹参见美国器官获取与移植网络关于配对捐献的介绍。

²参见 Dickerson 等, *Position-Indexed Formulations for Kidney Exchange*, 2016, <https://arxiv.org/abs/1606.01623>。

根据之前描述的算法，其具体运行过程如下：

1. 男生 1 向自己的首选 a 申请。 a 还没有对象，暂时接受他，得到 $(1, a)$ 。
2. 男生 2 向自己的首选 b 申请。 b 也没有对象，暂时接受他，此时有 $(1, a)$ 和 $(2, b)$ 两对。
3. 男生 3 向 a 申请。 a 更喜欢 3，因此改为接受 3，并让原来的男生 1 回到队尾。暂时配对变为 $(3, a)$ 和 $(2, b)$ 。
4. 轮到男生 1。他已被 a 拒绝，于是向下一位 b 申请。 b 更喜欢 1，因此用 1 替换 2，男生 2 回到队尾。暂时配对变为 $(3, a)$ 和 $(1, b)$ 。
5. 男生 2 接着向 a 申请。 a 更喜欢 2，因此用 2 替换 3，男生 3 回到队尾。暂时配对变为 $(2, a)$ 和 $(1, b)$ 。
6. 男生 3 接着向 b 申请。 b 更喜欢当前的男生 1，因此拒绝 3，现有配对不变，男生 3 回到队尾。
7. 男生 3 向最后一位 c 申请。 c 还没有对象，暂时接受他。此时队列为空，所有人都已配成对，算法结束。

最终结果是 $(1, b)$ 、 $(2, a)$ 、 $(3, c)$ 。男生 1 虽然更喜欢 a ，但 a 更喜欢当前的男生 2；男生 2 虽然更喜欢 b ，但 b 更喜欢当前的男生 1；男生 3 更喜欢的 a, b 也都更喜欢各自的当前对象，所以没有阻塞对。

定理 9.19 (延迟接受算法的正确性). 在双方人数相等、偏好表完整且没有并列的条件下，延迟接受算法总能得到稳定匹配。

证明. 先证明，只要还有男生没有对象，他就一定还能继续申请。由于男女各有 n 人，故此时也一定有女生没有对象。注意，一位女生只要收到过申请，就会暂时接受其中一人，以后即使更换对象，也始终有人与她配对。所以，目前没有对象的女生从未收到过申请，没有对象的男生自然也还没有向她申请过，他的偏好表就还没有用完。因此，只要尚未完成全部配对，算法就能继续处理下一次申请。而总共至多有 n^2 次申请，所以算法不可能无限进行，最终一定会让所有人都配成对。

再证明所得匹配稳定。假设存在阻塞对 (m, w) ，则男生 m 喜欢 w 胜过自己的最终对象。他按喜好依次申请，所以此前一定向 w 申请过，并被她拒绝。这意味着 w 自己选择匹配了比 m 更喜欢的人，之后也只会换成更喜欢的对象，所以最后不可能更喜欢 m ，与阻塞对的条件矛盾。□

当然，我们也可以让女生申请，男生暂时接受，同样能得到稳定匹配，证明完全对称，但匹配结果可能不同。

稳定匹配的一项实际应用，是安排医学生毕业后的住院医师培训岗位。在美国，申请者会根据专业方向、培训条件等给项目排序，各培训项目也会对申请者排序，而且每个项目有一定的招收名额。美国的住院医师匹配计划 (NRMP) 将双方的偏好集中起来，用匹配算法统一安排去向。它采用申请者发起申请的方式，基本过程与上面的延迟接受相同。申请者先尝试自己最喜欢的项目，项目在名额内暂时保留更喜欢的申请者，被拒绝的人再尝试后面的项目。与男女一一配对相比，一个项目可以接收多人，而且双方只把愿意接受的对象列入偏好表，因此也可能有人最终没有匹配到岗位。³

从房屋交换、肾脏捐献到培训岗位分配，匹配理论为这些实际制度的设计提供了方法。Alvin Roth 和 Lloyd Shapley 因对稳定分配理论与市场设计实践的贡献，共同获得了 2012 年诺贝尔经济学奖。⁴

³参见 NRMP 匹配算法介绍。

⁴参见 2012 年诺贝尔经济学奖公告。

9.6 补充材料：哈夫曼编码与信息熵

下面证明正文中提到的编码信息熵界。设字符至少有两种，各自的出现概率均为正，哈夫曼编码的平均码长为 L_H ，则

$$H(X) \leq L_H < H(X) + 1.$$

证明分为两步，先说明任何前缀码的平均码长都不小于熵，再构造一种平均码长小于熵加 1 的前缀码。哈夫曼编码既然最优，也就满足这个上界。

9.6.1 Kraft 不等式

设各码长为 $l_1, \dots, l_m$ ，最大码长为 D 。把编码树补成深度为 D 、每层都填满的二叉树，最下面一层有 2^D 个位置。长度为 l_i 的码字对应第 l_i 层的一个结点，从它向下延伸到第 D 层，共有 2^{D-l_i} 个位置。由于任何码字都不是另一码字的前缀，这些结点互不为祖先，它们延伸到下面的位置也不会重叠，因此 $\sum_i 2^{D-l_i} \leq 2^D$ 。进一步有 $\sum_i 2^{-l_i} \leq 1$ ，这就是二进制前缀码的 Kraft 不等式。

这个不等式还给出了构造前缀码的条件。如果正整数 $l_1, \dots, l_m$ 满足 $\sum_i 2^{-l_i} \leq 1$ ，就能按这些长度分配码字。为了理解这一点，先按长度递增处理各个码字。假设前 $j-1$ 个已经放好，现在要在第 l_j 层为下一个码字找位置。以前选过的结点及其后代都不能再用，否则旧码字就会成为新码字的前缀。第 i 个旧码字排除了这一层的 $2^{l_j-l_i}$ 个位置，合计为 $\sum_{i < j} 2^{l_j-l_i}$ 。由 Kraft 不等式， $\sum_{i < j} 2^{l_j-l_i} + 1 \leq 2^{l_j}$ ，所以这一层至少还剩一个位置，可以放入新码字。它的长度不短于已有码字，又避开了已有结点及其后代，因此不会产生前缀冲突。依次放下所有码字，就完成了构造。

9.6.2 下界证明

设字符 i 的概率为 $p_i > 0$ ，码长为 l_i ，平均码长为 $L = \sum_i p_i l_i$ 。要证明 $L \geq H(X)$ ，可以先把两者相减，整理成

$$H(X) - L = \sum_i p_i \log_2 \frac{2^{-l_i}}{p_i}.$$

注意对任意 $t > 0$ ，有 $\ln t \leq t - 1$ 。将 $t = 2^{-l_i}/p_i$ 代入，再乘以 p_i 并求和，得到

$$H(X) - L \leq \frac{1}{\ln 2} \sum_i p_i \left(\frac{2^{-l_i}}{p_i} - 1 \right) = \frac{1}{\ln 2} \left(\sum_i 2^{-l_i} - \sum_i p_i \right) \leq 0.$$

最后一步用到了 Kraft 不等式和 $\sum_i p_i = 1$ ，因此 $L \geq H(X)$ 。由于对数不等式只在 $t = 1$ 时取等号，所以平均码长等于熵，当且仅当每个字符都满足 $2^{-l_i} = p_i$ 。

9.6.3 上界证明

刚才的等号条件给出了理想的码长 $-\log_2 p_i$ ，只是它通常不是整数。一个直接的办法是把它向上取整，令 $l_i = \lceil -\log_2 p_i \rceil$ ，这样每个码字只比理想长度多出不到一位。

还要确认这些长度确实能构成前缀码。因为 $l_i \geq -\log_2 p_i$ ，所以 $2^{-l_i} \leq p_i$ ，相加后得到 $\sum_i 2^{-l_i} \leq 1$ 。由 Kraft 不等式的逆命题，存在一组使用这些长度的前缀码，其平均码长满足

$$\sum_i p_i \lceil -\log_2 p_i \rceil < \sum_i p_i (-\log_2 p_i + 1) = H(X) + 1.$$

哈夫曼编码的平均码长不大于刚构造的前缀码，因此也小于 $H(X) + 1$ 。

9.7 习题

习题 9.1 (加权活动选择). 给定 n 个活动, 活动 j 占用时间 $[s_j, f_j)$, 收益为 $v_j \geq 0$, 其中 $0 \leq s_j < f_j$ 。同一时刻只能参加一个活动。请设计动态规划算法, 求最大总收益及一组达到该收益的活动。

习题 9.2 (跳跃游戏). 给定非负整数数组 $nums[0..n-1]$, 其中 $n \geq 1$ 。你最初位于下标 0, 在位置 i 可以向右跳不超过 $nums[i]$ 格, 目标是到达下标 $n-1$ 。

1. 设计 $O(n)$ 时间算法, 判断能否到达终点。
2. 若可以到达, 求最少跳数, 时间仍要求为 $O(n)$ 。

例如 $nums = [2, 3, 1, 1, 4]$ 时, 可以按 $0 \rightarrow 1 \rightarrow 4$ 跳两次到达终点; $nums = [1, 0, 2]$ 时则无法到达。

习题 9.3 (贪心策略设计). 判断下面两种安排是否总能达到目标, 并说明理由。

1. 一台机器从时刻 0 开始依次处理若干任务, 每项任务的处理时间和截止日期都已知, 任务开始后不能中断。任务的完成时间超过截止日期多少, 就记多少延迟, 按时完成则记延迟为 0。如果希望所有任务的延迟加起来尽量小, 按截止日期从早到晚执行, 是否总是最优的?
2. 要给一组活动安排教室, 时间重叠的活动不能使用同一间教室, 目标是尽量少用教室。将活动按结束时间从早到晚处理。对于当前活动, 如果有教室在它的整个活动时间内都空闲, 就任选一间使用, 否则增加一间。这种方法是否总能使用最少的教室?

习题 9.4 (离线最优缓存). 程序需要依次访问一些数据页, 每页大小相同, 缓存中最多能放 $k \geq 1$ 页。如果要访问的页面已经在缓存中, 就可以直接使用; 如果不在, 就必须把它读入缓存, 这称为一次缺页。缓存已满时, 还要先选一页移出, 为新页面腾出位置。只在缺页时读入页面。

现在已知缓存最初存放的页面集合 C_0 , 其中 $|C_0| \leq k$, 也提前知道程序接下来要访问的完整序列 $r_1, \dots, r_N$ 。请设计算法, 决定每次缓存已满时移出哪一页, 使整个访问过程的缺页次数最少, 并证明其正确性、分析时间复杂度。

例如, 缓存能放两页, 最初存着 a, b , 接下来依次访问 a, b, c, b, c, a, b 。前两次访问都能直接使用缓存, 访问 c 时则必须在 a, b 中选一页移出。这个选择会影响后面是否还要重新读入被移出的页面。

习题 9.5 (分数背包问题). 假如背包中的物品可以分割, 例如粮食或金属, 装不下整件时也可以只取一部分, 其价值按所取重量的比例计算。现在有 n 件物品, 物品 i 的重量为 $w_i > 0$, 价值为 $v_i \geq 0$, 背包容量为 $C \geq 0$ 。怎样选取, 才能使装入的总价值最大?

1. 设计一个解决这一问题的贪心算法, 并计算其时间复杂度;
2. 证明你设计的贪心算法能返回问题的全局最优解;
3. 你设计的贪心算法对 0-1 背包问题是否适用? 如果适用请给出证明, 不适用请举出反例。

习题 9.6 (硬币找零). 假设每种面额的硬币都有无限枚, 要用尽量少的硬币凑出给定的非负整数金额。当面额为 $1, c, c^2, \dots, c^k$ (整数 $c \geq 2$) 时, 证明每次尽量取最大面额的贪心算法正确。再举出一组包含面额 1 的硬币, 说明这种方法对一般面额不一定正确。

习题 9.7 (哈夫曼编码). 以下均采用二进制哈夫曼编码, 请回答下列问题并说明理由。

1. 一棵哈夫曼树共有 61 个结点, 它为多少种字符编码? 哈夫曼树的总结点数可能是 60 吗?
2. 为 n 种字符构造哈夫曼编码时, 最长码长一定不超过 $\lceil \log_2 n \rceil$ 位吗?

3. 八种字符的频数分别为 1, 1, 2, 3, 5, 8, 13, 21, 其哈夫曼编码的最长、最短码长分别是多少? 最多有几种字符的码长相同?

习题 9.8 (田忌赛马). 田忌和齐王各有 n 匹马, 每匹马的速度已知, 且只能出场一次。每场比赛各派一匹马, 速度快者获胜。从田忌的角度, 获胜得 1 分、失败得 -1 分、平局得 0 分。田忌可以安排双方的配对, 怎样使 n 场比赛的总得分最大? 请给出算法、正确性证明和复杂度。

例如, 双方马速分别为 [1, 2, 3] 和 [2, 3, 4]。按速度逐一配对会输掉三场; 改为 1 对 4、2 对 3、3 对 2, 总得分可以提高到 -1。

习题 9.9 (最大间隙聚类). 本题希望把 n 个点分成 k 个非空类别, 让不同类别之间尽量远离。已知任意两点间的非负距离, 且 $2 \leq k \leq n$ 。对于一种划分, 考察所有分属不同类别的点对, 把其中最短的距离称为该划分的间隙值, 我们的目标是让这个值尽量大。请设计贪心算法, 证明其正确性并分析复杂度。如果先求一棵最小生成树, 再删去其中最长的 $k-1$ 条边, 将剩余各棵树中的点分别归为一类, 是否也能得到最优划分? 请说明理由。

9.8 参考答案

习题 9.1. 与非加权版本递推式类似, 区别在于用活动收益 v_j 代替计数。 $\square$

习题 9.2. 第一问, 可以从左向右检查每个位置能把可达范围扩展到哪里。用 far 记录目前能到达的最远下标, 初始为 0。只要 $i \leq far$, 就能到达位置 i , 从这里还可以跳到 $i + nums[i]$, 所以更新

$$far = \max\{far, i + nums[i]\}.$$

只要 $far \geq n-1$, 就能到达终点。如果检查到 $i > far$, 说明前面所有可达位置都跳不到这里, 后面的位置也就无从到达。

第二问, 依次求出至多跳 0 次、1 次、2 次能到达的最远位置。设至多跳 q 次能到达 $0..end$, 那么从这些位置再跳一次, 最远能到达

$$far = \max_{0 \leq i \leq end} \{i + nums[i]\}.$$

这就是至多跳 $q+1$ 次能到达的最远位置。如果第一次在这一轮得到 $far \geq n-1$, 最少跳数就是 $q+1$ 。如果 $far = end < n-1$, 说明再跳一次也不能到达任何新位置, 终点不可达。

第一问显然只需遍历一次数组, 第二问实现时下一轮只需检查新到达的位置, 将它们能跳到的最远下标与 far 比较, 无需重扫旧位置, 这样每个下标只检查一次。因此两问时间均为 $O(n)$ 。 $\square$

习题 9.3. 第一问不能保证最优。取两个任务, 处理时间和截止日期分别为 $(t, d) = (10, 1), (1, 2)$ 。按截止日期执行时, 两项任务的延迟分别为 9、9, 总和为 18。先执行短任务时, 延迟分别为 0、10, 总和只有 10。正文的相邻交换证明保证了最大延迟不会增加, 对延迟之和没有这样的保证。

第二问也不能保证最优。取 $a = [0, 2), b = [1, 4), c = [4, 5), d = [2, 6)$, 结束顺序为 a, b, c, d 。先把 a, b 分到两间教室, 再把 c 放入原来举办 a 的教室, 则 d 与该教室的 c 冲突, 也与另一教室的 b 冲突, 需要第三间。最优分配是 a, d 同一间、 b, c 同一间, 只需两间。 $\square$

习题 9.4. 缓存满时, 应优先留下很快又会用到的页面, 因此换出下一次访问最晚的那一页, 这称为“将来最远”策略。如果某页以后不再使用, 就把它下次访问时间视为无穷远。用交换参数法证明贪心选择性质。取一个最优方案 O , 找到它第一次与贪心选择不同的换页。此前两种方案的缓存相同, 假设这次贪心换出 x , 而 O 换出 y 。我们把 O 改为换出 x , 得到方案 O' , 再调整后面的操作, 证明缺页次数不会增加。注意这次换页后, O' 比 O 多存一页 y 、少存一页 x , 其余页面相同。因为贪心选择了 x , 所以 y 一定先于 x 被再次访问, 或者两者以后都不再访问。在此之前, 访问其他页面时, 两种方案会同时命中或同时缺页, 可以作出相同的换页选择。需要单独处理的只有以下两种情况。

1. O 在一次缺页时换出 x 。此时让 O' 换出 y , 再调入同一页面, 两份缓存就完全相同, 累计缺页次数也相同。
2. 再次访问 y 。这时 O 缺页, O' 命中, 因而 O' 少花一次调入的代价。如果 O 换出 x 来装入 y , 两份缓存直接变得相同。否则, 假设 O 换出 z , 就暂时让 O' 也换出 z , 并提前调入 x 。这多花的一次调入恰好抵消刚才省下的一次, 两份缓存再次相同, 总代价没有增加。

第二种情况暂时允许了提前调入, 而题目只允许缺页时调入。可以把这次操作推迟到真正访问 x 时再做, 等待期间保留一个空位即可。如果上述两种情况都没有发生, x, y 就一直没有被再次访问, 两种方案始终有相同的缺页次数。这样总能得到一个在当前选择上与贪心一致的最优方案。对后续分歧重复调整, 就证明了贪心算法最优。

要实现这个策略, 关键是知道缓存中的每一页下一次会在什么时候使用。我们先从后向前扫描访问序列, 用一个数组记住各页下一次最近被扫描到的位置。为了快速找出最晚才会再次使用的页面, 把缓存中的页面放入最大堆, 以下次访问的位置为键。缓存已满且发生缺页时, 删除堆顶, 再放入新读入的页面即可。如果缓存命中, 也需要更新相应页面下一次出现的时间。设共有 N 次访问, 初始缓存的页面集合为 C_0 , 缓存最多能放 k 页。假设涉及的页面已连续编号, 可以直接用编号访问上述数组。初始化数组和逆序预处理共需 $O(N + |C_0|)$ 时间, 初始建堆需要 $O(|C_0|)$ 时间。此后每次访问只需常数堆操作, 而堆中至多有 k 页, 所以总时间为 $O(|C_0| + N \log k)$ 。□

习题 9.5. 算法与正确性证明见例 8.4, 时间复杂度为 $O(n \log n)$ 。贪心算法不适用于 0-1 背包: 假设容量为 50, 三件物品的(重量, 价值)为 (10, 60), (20, 100), (30, 120), 按单位重量价值递减的策略先取前两件, 价值 160, 但最优是选择后两件, 价值 220。□

习题 9.6. 最优解中, 每种非最大面额 c^i 的硬币都少于 c 枚, 否则可用一枚 c^{i+1} 替换 c 枚, 枚数严格减少。注意 $(c-1)(1+c+\cdots+c^{k-1})=c^k-1$, 因此在最优解中, 所有较小面额加起来凑不出一枚 c^k , 所以给定金额中每一个完整的 c^k 只能由最大面额支付。 c^k 的枚数恰好是金额除以 c^k 的整数商。剩余金额小于 c^k , 再对下一种面额重复这个论证, 就得到了逐次尽量取大面额的贪心方案。

对面额 1, 3, 4、金额 6, 贪心得到 $4+1+1$ 三枚, 最优是 $3+3$ 两枚。□

习题 9.7. (1) 设二叉哈夫曼树有 n 个叶子和 I 个内部结点。每个内部结点连接两个孩子, 所以树中有 $2I$ 条边。另一方面, 树的边数比结点数少 1, 也就是 $I+n-1$, 因此 $2I=I+n-1$, 解得 $I=n-1$ 。总结点数为 $2n-1$, 所以 61 个结点对应 31 个字符。总结点数不可能为 60, 因为二叉哈夫曼树结点数必须是奇数; (2) 不一定: 四个字符频数为 1, 1, 2, 4 时, 合并权重为 2, 4, 8, 码长为 3, 3, 2, 1, 最长 3 位, 大于 $\lceil \log_2 4 \rceil = 2$; (3) 最长码长为 7, 最短码长为 1, 最多有两种字符的码长相同。□

习题 9.8. 先将双方的马按速度排序。每次安排比赛时, 如果田忌最快的马能赢齐王最快的马, 就让它们比赛, 先赢下一场。如果田忌最快的马比齐王最快的马慢, 那么齐王这匹马无论遇到谁都会赢,

就派田忌最慢的马去输掉这一场。如果双方最快的马一样快，就再比较双方最慢的马。田忌最慢的马能赢时，让两匹最慢的马比赛。否则，用田忌最慢的马对齐王最快的马。每安排完一场，就从两边删去出场的马，继续按同样的方法选择。排序需要 $O(n \log n)$ 时间，随后每场比赛只需常数比较，共需 $O(n)$ 时间。使用交换参数法不难证明每一种贪心选择都包含在某个最优配对中，而最优子结构性质显然成立，故算法正确性得证。 $\square$

习题 9.9. 要让不同类别之间的最短距离尽量大，可以先把距离近的点合在一起。将点看作顶点，任意两点之间连边，边权就是距离。开始时每个点各自一类，按边长从小到大检查，如果一条边的两端分属不同类别，就把这两类合并，直到剩下 k 类。这就是提前停止的 Kruskal 算法，所得森林中的每棵树对应一个类别。设算法所得划分为 C ，间隙值为 λ 。如果继续执行 Kruskal，下一条将要选入的边就是连接不同类别的最短边，所以它的长度为 λ ，之前已经选入的边都不长于它。

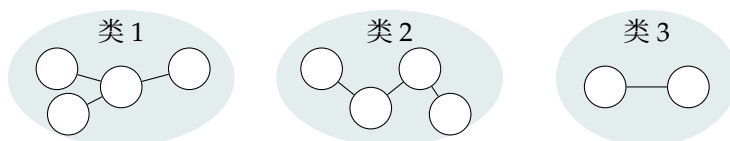


图 9.7 剩下三棵树时停止合并，每棵树中的点归为一类。

取任意一个不同的 k 类划分 C' 。 C 中至少有一类的点被 C' 分到了不同类别。取这样两个被分开的点，它们在 C 中由一条已选边组成的路径连接。沿路径从一个点走到另一个点，至少有一条边的两端属于 C' 的不同类别，而这条边不长于 λ ，由此可证 C 最优。至于时间复杂度，注意共有 $O(n^2)$ 条边，排序及并查集操作需要 $O(n^2 \log n)$ 时间。

先求最小生成树，再删去最长的 $k-1$ 条边，也能得到最优划分，理由与上面类似，不再赘述。 $\square$

9.9 参考资料

1. 浙江大学《高级数据结构与算法分析》课程课件第九讲。
2. Cormen、Leiserson、Rivest、Stein, *Introduction to Algorithms*, 第三版, 第 16 章介绍活动选择、分数背包、哈夫曼编码、拟阵与单位时长任务调度, 思考题 16-1、16-5 分别讨论找零与离线缓存。
3. Kleinberg、Tardos, *Algorithm Design*, 第 1.1、4.1–4.3、4.7–4.8 节分别介绍稳定匹配、教室分配、任务延迟、最优缓存、聚类及哈夫曼编码。
4. Huffman, D. A. *A Method for the Construction of Minimum-Redundancy Codes*. *Proceedings of the IRE*, 40(9):1098–1101, 1952. <https://doi.org/10.1109/JRPROC.1952.273898>.
5. Roughgarden, T. *CS364A: Algorithmic Game Theory*, 2013, Lecture 9, Section 3 介绍住房分配及首选交易环, Lecture 10 介绍稳定匹配。 <https://theory.stanford.edu/~tim/f13/f13.html>.